Modern LLMs

From Decoder-Only Transformers to GPT-3 and Beyond

ModIA 5 - Advanced Artificial Intelligence

Paul Novello & David Bertoin

February 23, 2026

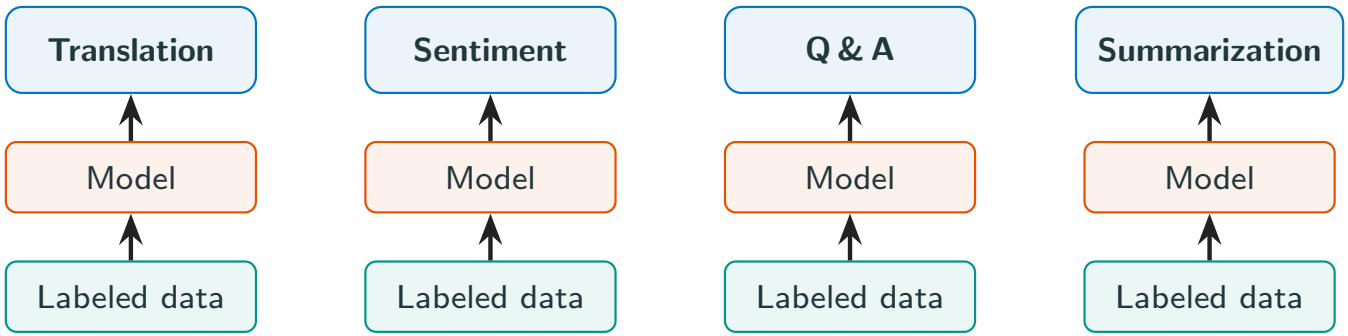
Outline

- → Motivation & Background
- → Decoder-Only Transformers
- → Modern LLMs

Motivation & Background

The NLP Landscape Before LLMs

Before LLMs, every task required its own **labeled dataset** and **dedicated model**.



No knowledge transfer between tasks

Labels are **expensive**, scarce, and domain-specific

Billions of unlabeled web pages — completely unused

The Self-Supervised Insight: Next-Token Prediction

Key idea: predict the next token given all previous tokens — **no labels needed**.

$$\mathcal{L}(\theta) = -\sum_{t=1}^{n} \log p_\theta(x_t \mid x_{<t})$$

→ Any text is its own label — the **whole internet** becomes training data

→ Zero human annotation required

→ Scales freely: more data + more compute ⇒ better models

Self-Supervised Learning at Scale

No labeling bottleneck $\Rightarrow$ train on the **entire web**.

Data sources used in practice:

- $\rightarrow$ Common Crawl (petabytes of web text)
- $\rightarrow$ Wikipedia & encyclopedias
- $\rightarrow$ Books, news, social media
- $\rightarrow$ GitHub (billions of lines of code)
- $\rightarrow$ ArXiv, PubMed (science)
- $\rightarrow$ Multilingual corpora

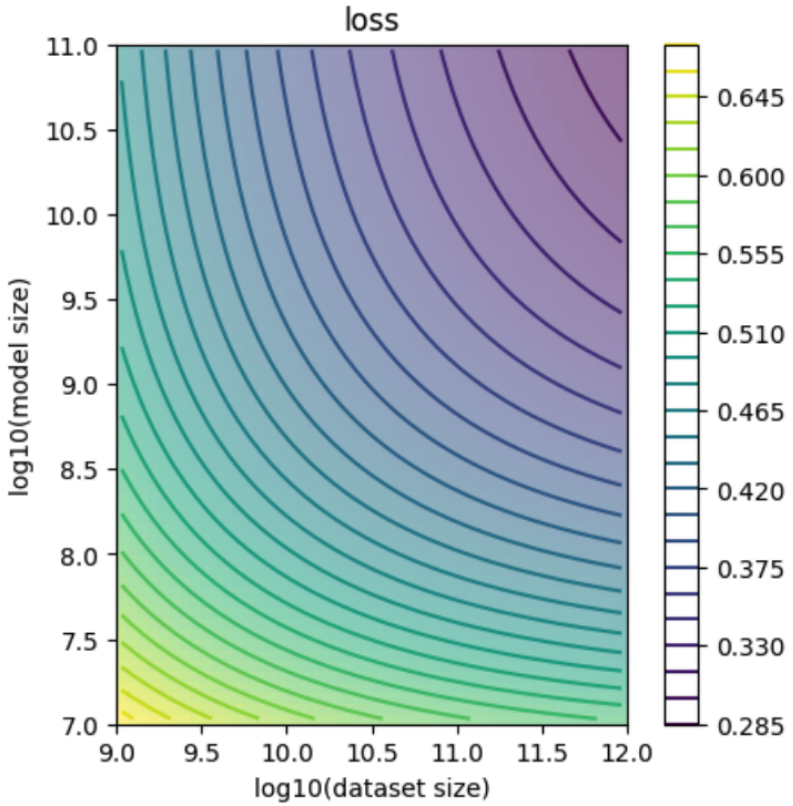
Scaling Laws

Loss follows **power laws** (Kaplan et al., 2020; Hoffmann et al., 2022):

$$L(N) \propto N^{-\alpha_N}$$
$$L(D) \propto D^{-\alpha_D}$$

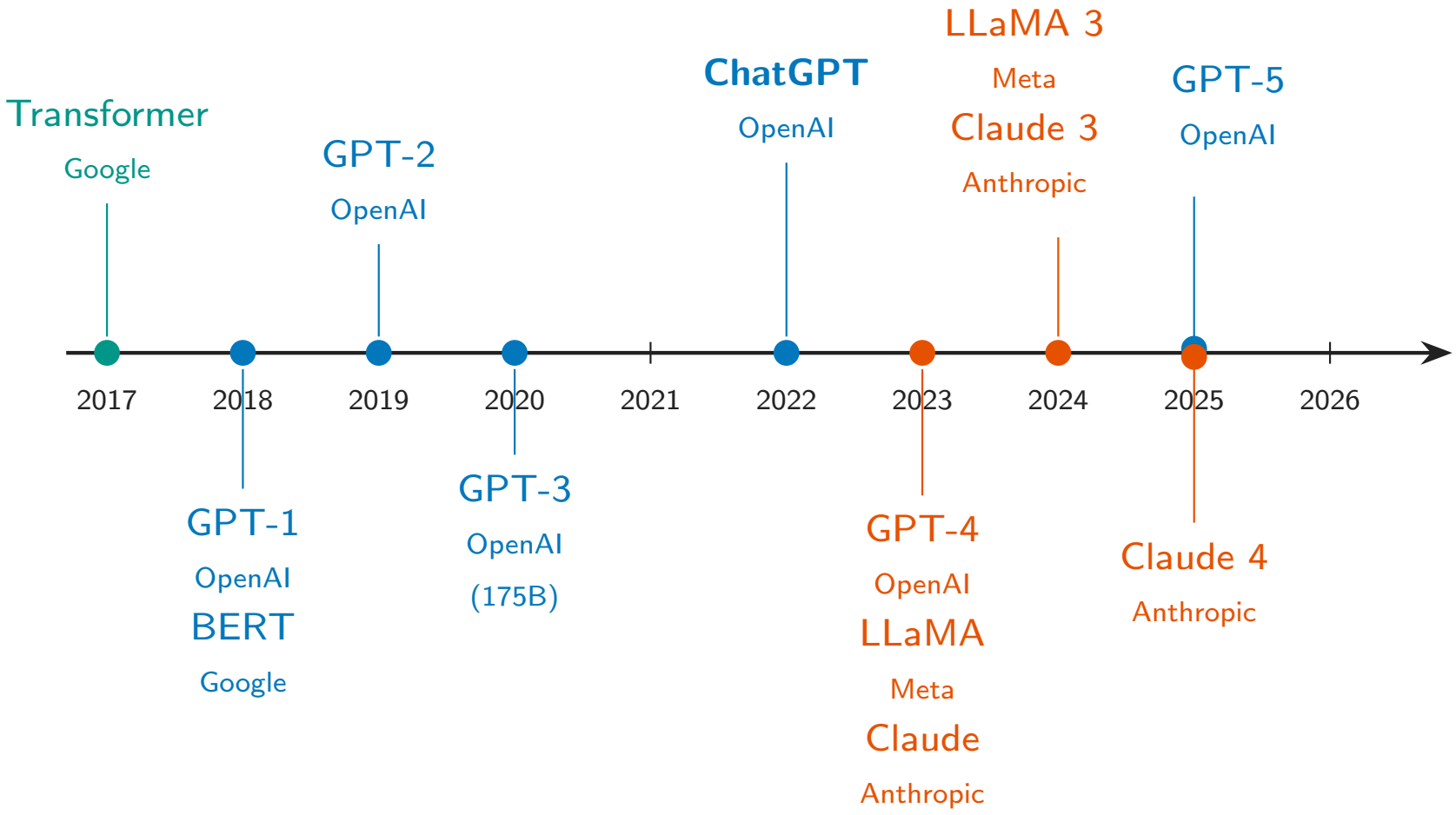
Optimal "**Chinchilla scaling**" ($C =$ compute budget):

$$N_{\text{opt}} \propto C^{0.5}, \qquad D_{\text{opt}} \propto C^{0.5}$$



$N =$ parameters, $D =$ training tokens, $C \approx 6ND$ FLOPs.

The Rise of Large Language Models



OpenAI Meta ANTHROP\C

LLMs Change the World

ChatGPT reached

100M

users in just 60 days

Platform	Time to 100M
ChatGPT	2 months
TikTok	9 months
Instagram	30 months

Applications across domains:

→ Code generation & debugging

→ Document summarization

→ Scientific research

→ Healthcare assistance

→ Education & tutoring

→ Creative writing

Decoder-Only Transformers

Decoder-Only Architecture

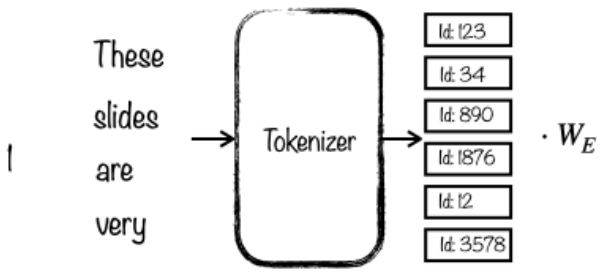
Key simplification: remove the encoder entirely (GPT; Radford et al., 2018).

Only self-attention (no cross-attention)
Causal masking: each token attends only to previous tokens
Autoregressive: $p(x_1, \ldots, x_n) = \prod_{t=1}^{n} p(x_t \mid x_{<t})$

This is the architecture behind GPT, LLaMA, Claude, and most modern LLMs.

Vaswani et al. (2017) introduced the encoder–decoder Transformer. Radford et al. (2018) showed that the decoder alone suffices for language modeling.

Tokenization: BPE



Tokenization: BPE

Byte-Pair Encoding (Sennrich et al., 2016):

→ Start with character-level vocabulary
→ Count all adjacent symbol pairs in corpus
→ Merge most frequent pair into a new symbol
→ Repeat until vocabulary size $|V|$ is reached

Example (merge `es` then `est`):

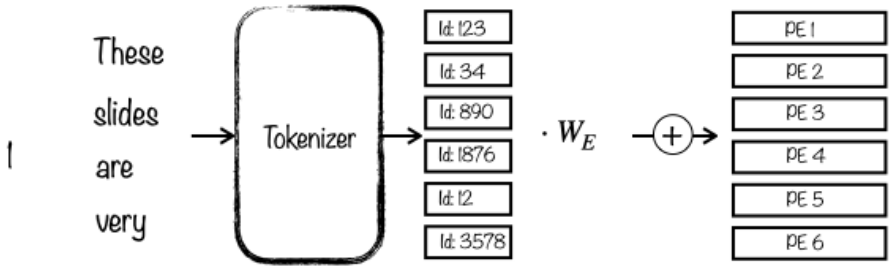
$$\texttt{n e w e s t} \mid \texttt{w i d e s t}$$

$$\xrightarrow{\text{merge } (e,s)} \texttt{n e w } \underline{\texttt{es}} \texttt{ t} \mid \texttt{w i d } \underline{\texttt{es}} \texttt{ t}$$

$$\xrightarrow{\text{merge } (es,t)} \texttt{n e w } \underline{\texttt{est}} \mid \texttt{w i d } \underline{\texttt{est}}$$

Typical vocabulary size: 32K–128K tokens. Each token gets a token ID

Positional Encoding



Positional Encoding

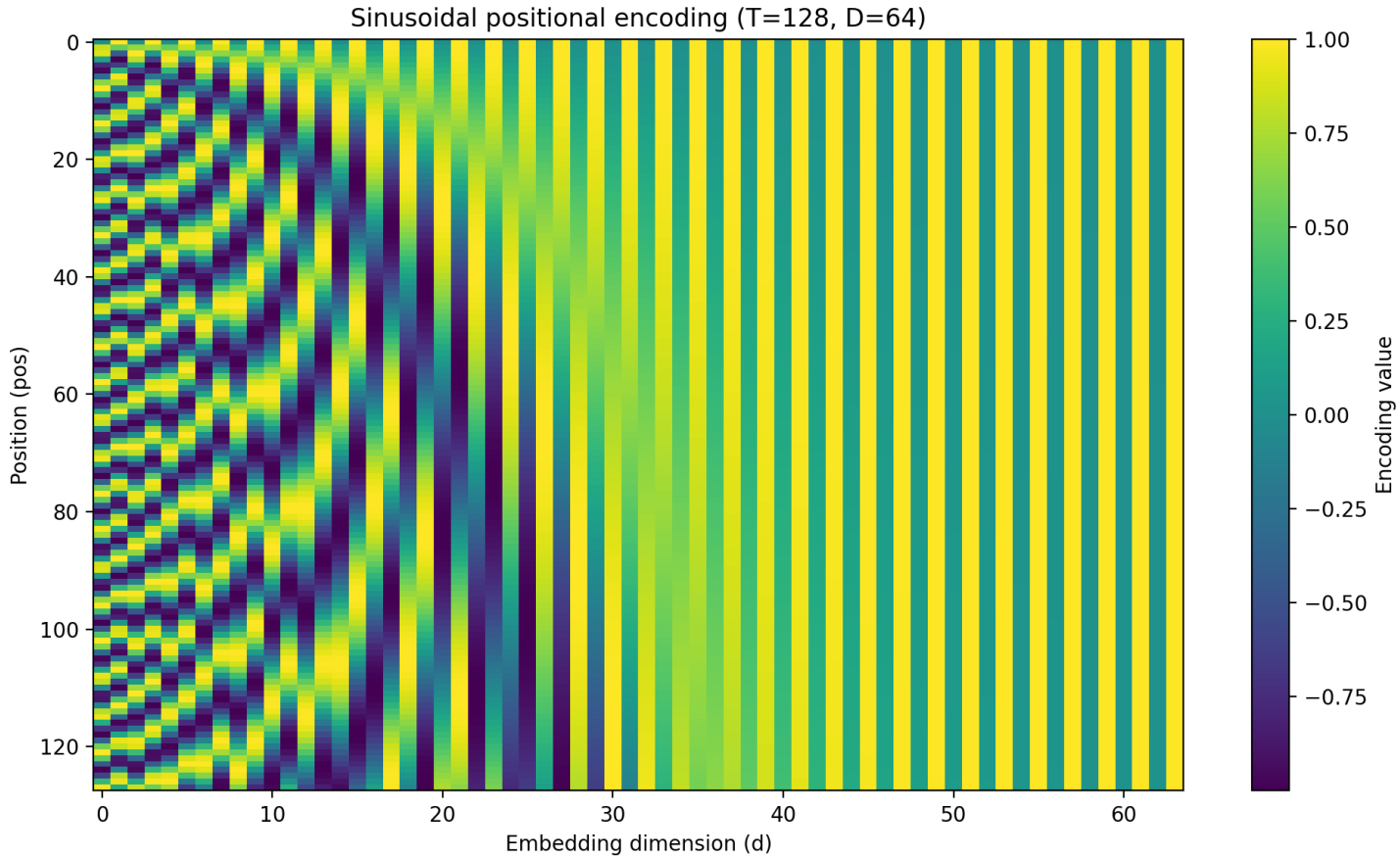
Self-attention is **permutation-invariant**: it has no notion of order. Causal mask is not enough — all tokens can attend to themselves and previous tokens, but they have no way to distinguish position.

Sinusoidal positional encoding (Vaswani et al., 2017): Each token at position pos gets a d-dimensional vector added ($+$) to its embedding:

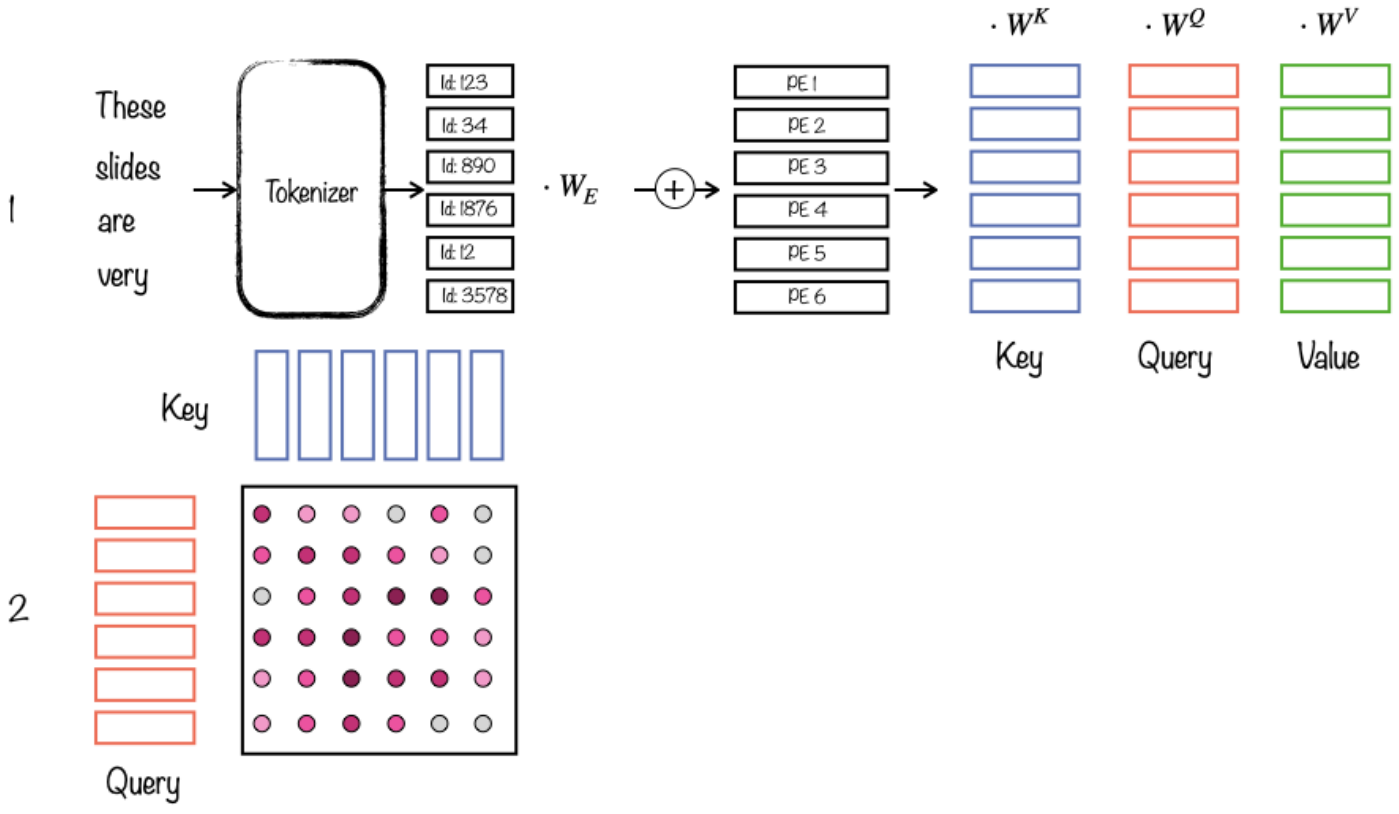
$$PE_{\text{pos}} = \big[\sin(\text{pos}/\omega_0),\ \cos(\text{pos}/\omega_0),\ \ldots,\ \sin(\text{pos}/\omega_{d/2-1}),\ \cos(\text{pos}/\omega_{d/2-1})\big]$$

where $\omega_k = 10000^{2k/d}$. Low-index dimensions oscillate fast (word-level), high-index dimensions oscillate slowly (paragraph-level).

Positional Encoding



Query, Key, Value



Query, Key, Value

Given input $X \in \mathbb{R}^{n \times d}$ (sequence of n token embeddings):

Linear projections:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

with $W^Q, W^K \in \mathbb{R}^{d \times d_k}$, $W^V \in \mathbb{R}^{d \times d_v}$.

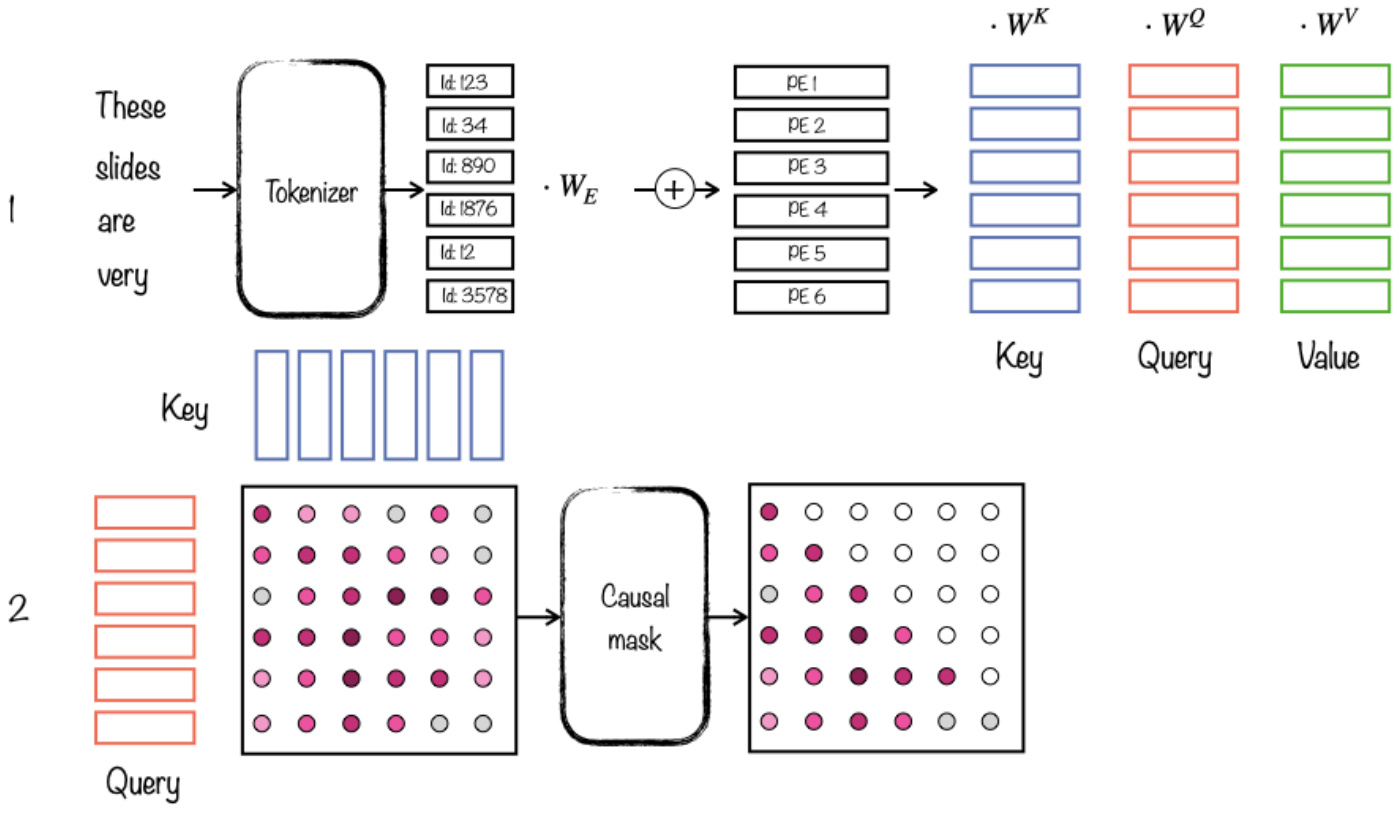
Intuition:

Query Q: "what am I looking for?"

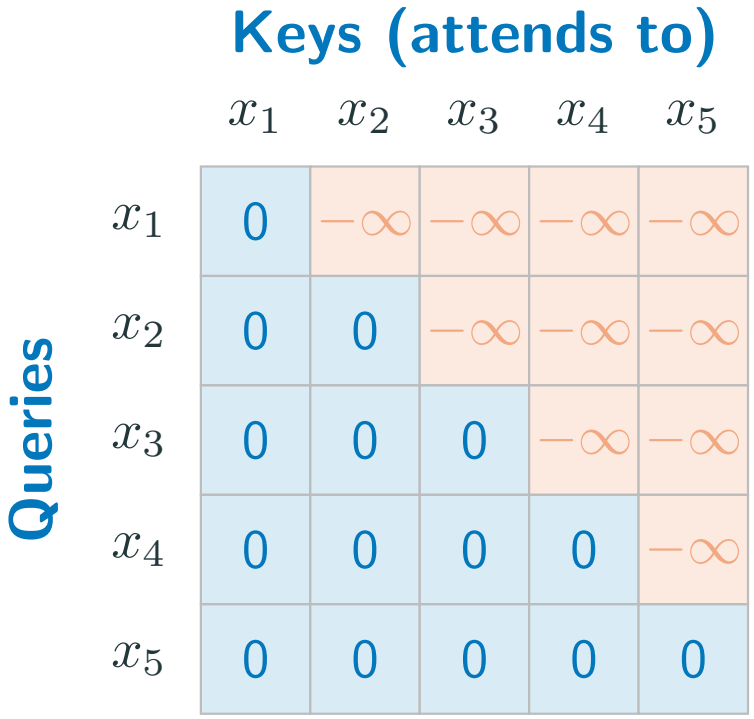
Key K: "what do I contain?"

Value V: "what information do I provide?"

Causal Mask


1
These
slides
are
very
Tokenizer
Id: 123
Id: 34
Id: 890
Id: 1876
Id: 12
Id: 3578
$\cdot W_E$
PE 1
PE 2
PE 3
PE 4
PE 5
PE 6
$\cdot W^K$
$\cdot W^Q$
$\cdot W^V$
Key
Query
Value
Key
2
Query
Causal
mask

Causal Mask



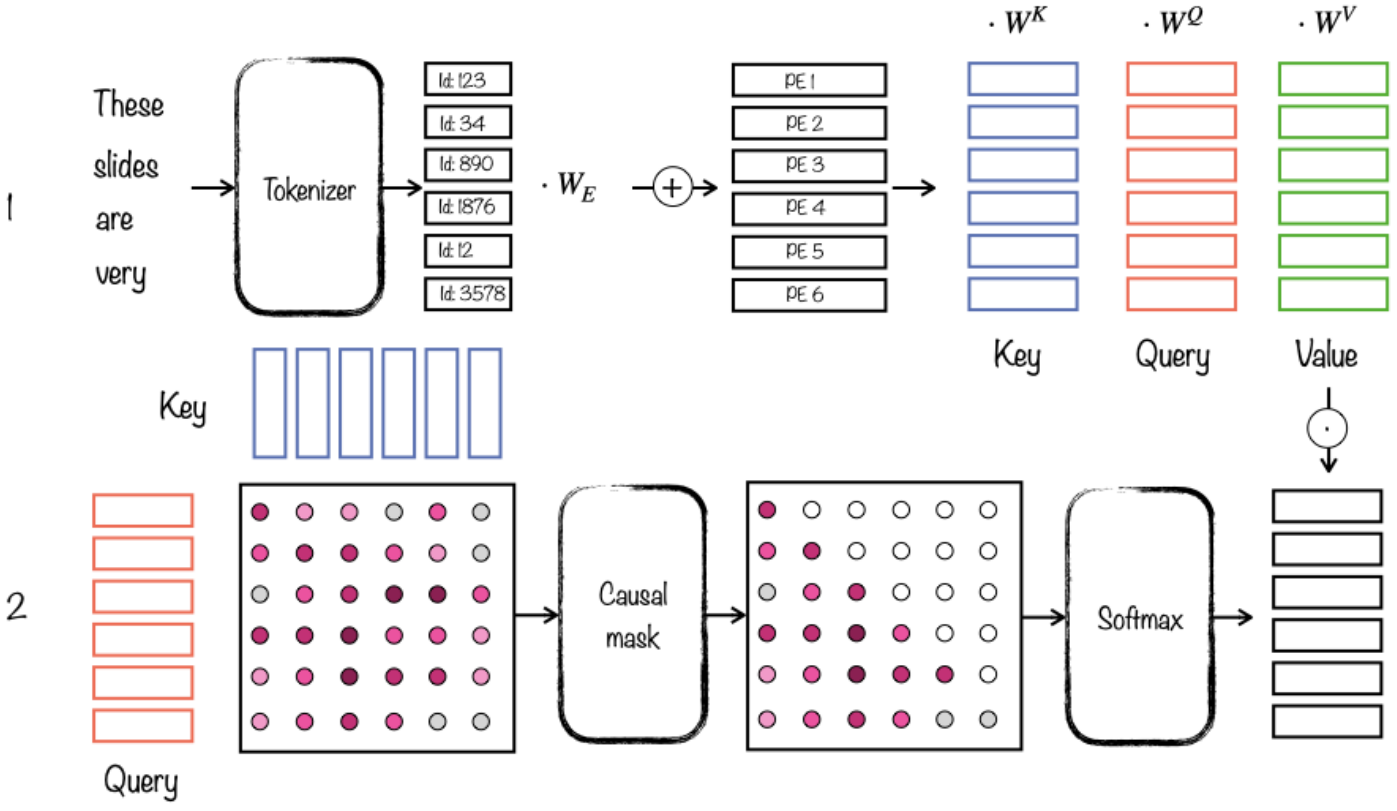
Masking via additive mask M before softmax:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

Each token can only attend to itself and previous tokens.

Future tokens are masked with $-\infty$ before softmax.

Scaled Dot-Product Attention



Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

Step by step:

- → Compute similarity: $S = QK^\top \in \mathbb{R}^{n \times n}$
- → Scale: $S \leftarrow S/\sqrt{d_k}$
- → Mask: $S \leftarrow S + M$ (causal mask)
- → Normalize: $A = \text{softmax}(S)$ (row-wise)
- → Aggregate: $\boldsymbol{h} = A\,V \in \mathbb{R}^{n \times d_v}$

Each row of A is a probability distribution over past and current positions.

Why Scale by $\sqrt{d_k}$?

Let $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{d_k}$ be one query/key row pair from Q, K. Assume components $q_i, k_i \overset{\text{iid}}{\sim} \mathcal{N}(0, 1)$. Then:

$$\boldsymbol{q}^\top \boldsymbol{k} = \sum_{i=1}^{d_k} q_i k_i \quad \Longrightarrow \quad \mathbb{E}[\boldsymbol{q}^\top \boldsymbol{k}] = 0, \;\; \text{Var}[\boldsymbol{q}^\top \boldsymbol{k}] = d_k$$

Without scaling, as d_k grows:

- Dot products become large in magnitude
- Softmax saturates $\rightarrow$ near-zero gradients

With scaling:

$$\text{Var}\left[\frac{\boldsymbol{q}^\top \boldsymbol{k}}{\sqrt{d_k}}\right] = 1 \qquad \text{regardless of } d_k$$

Multi-Head Attention

Run p attention heads in parallel, each with its own projections:

$$\boldsymbol{h}_i = \text{Attention}\big(QW_i^Q,\ KW_i^K,\ VW_i^V\big)$$

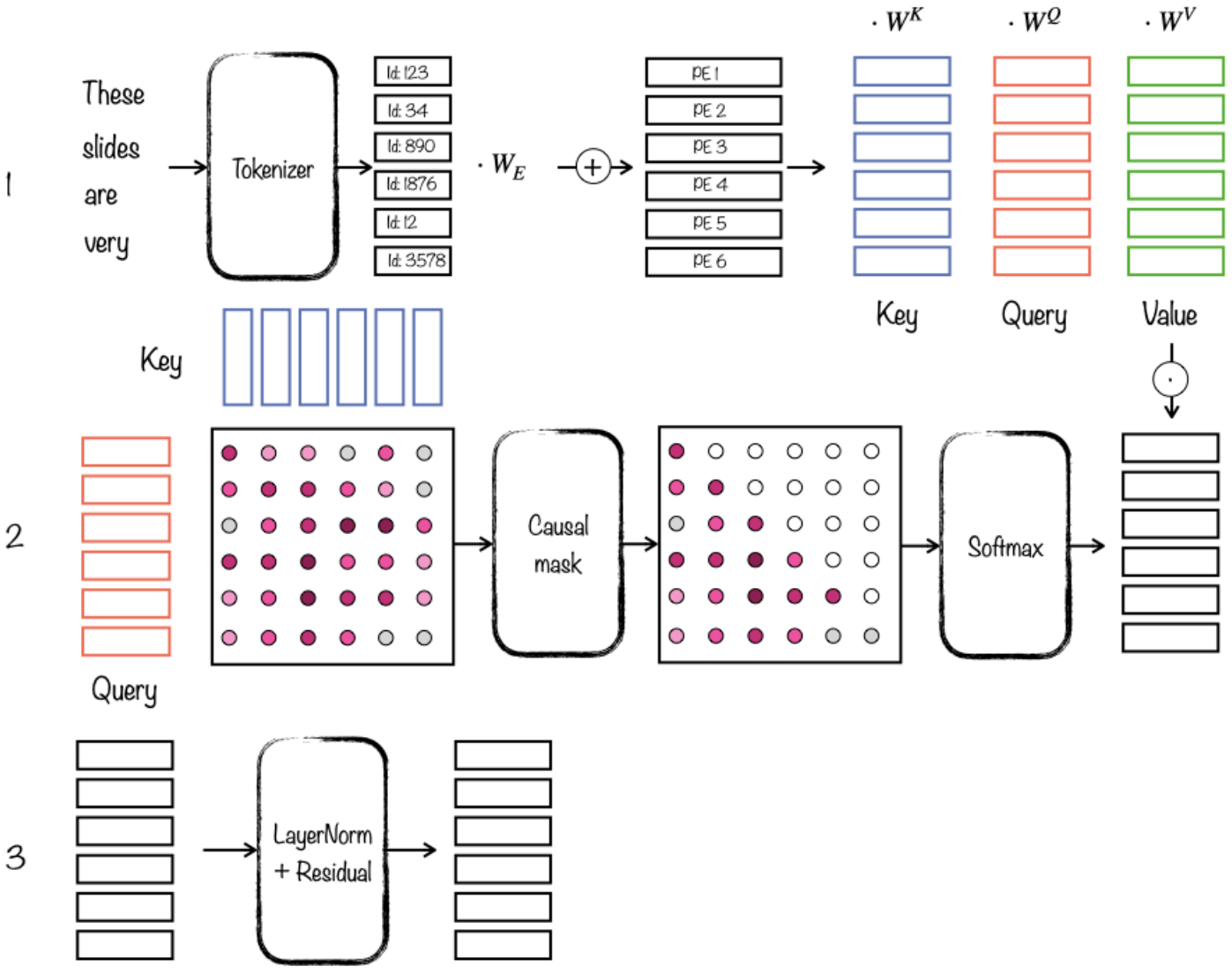
Concatenate and project:

$$\boldsymbol{h} = \text{MultiHead}(Q, K, V) = \text{Concat}(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_p)\, W^O$$

Dimensions: $W_i^Q, W_i^K \in \mathbb{R}^{d \times d_k}$, $W_i^V \in \mathbb{R}^{d \times d_v}$, $W^O \in \mathbb{R}^{pd_v \times d}$.

Typically $d_k = d_v = d/p$. Cost $\approx$ single full-dimension head.
So $\boldsymbol{h} \in \mathbb{R}^{n \times d}$ regardless of p.

MLP Layer


1
These
slides
are
very
Tokenizer
Id: 123
Id: 34
Id: 890
Id: 1876
Id: 12
Id: 3578
$\cdot W_E$
PE 1
PE 2
PE 3
PE 4
PE 5
PE 6
$\cdot W^K$
$\cdot W^Q$
$\cdot W^V$
Key
Query
Value
Key
2
Causal
mask
Softmax
Query
3
LayerNorm
+ Residual

MLP Layer

Layer Norm (Ba et al., 2016) — normalizes across features:

$$\text{LayerNorm}(\boldsymbol{h}) = \boldsymbol{\gamma} \odot \frac{\boldsymbol{h} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \boldsymbol{\beta}$$

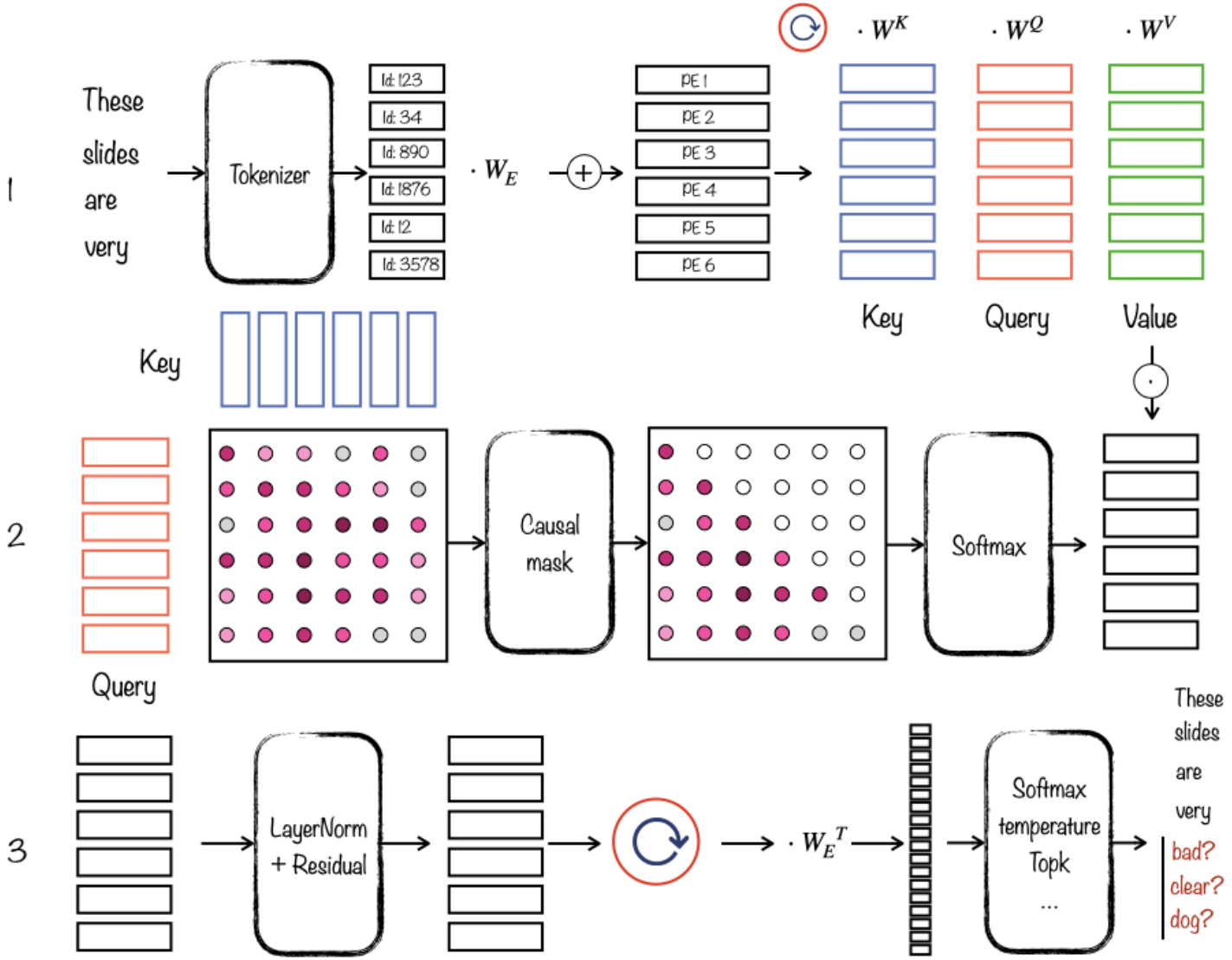
where:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} h_i, \qquad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(h_i - \mu)^2$$

Residual connection + LayerNorm:

$$\boldsymbol{h} = \text{LayerNorm}\Big(\boldsymbol{h} + \text{Layer}\big(\text{LayerNorm}(\boldsymbol{h})\big)\Big)$$

Logits and Softmax



Logits and Softmax

The previous operations can be stacked into multiple layers. After L layers, we get hidden states $\boldsymbol{h} \in \mathbb{R}^{n \times d}$ where $\boldsymbol{h}_t$ corresponds to the hidden state of position t.

$$\boldsymbol{h}_t \in \mathbb{R}^d.$$

Logits are a linear projection from hidden dimension d to vocabulary size $|V|$:

$$\boxed{\boldsymbol{z}_t = \boldsymbol{h}_t W_E^\top \in \mathbb{R}^{|V|}}$$

(with **weight tying**: same W_E as the input embedding matrix).

For next token prediction, we only use logits from the last position

Logits and Softmax

Softmax turns logits into a probability distribution over the vocabulary:

$$p(x_{t+1} = v \mid x_{\leq t}) = \text{softmax}(\boldsymbol{z}_t)_v = \frac{\exp(z_{t,v})}{\sum_{v'=1}^{|V|} \exp(z_{t,v'})}.$$

Here $v \in \{1, \ldots, |V|\}$ indexes the vocabulary: $z_{t,v}$ is the score for the v-th token in the vocabulary.

Generation & Sampling

Autoregressive generation: sample $x_t \sim p(\cdot \mid x_{<t})$ sequentially.

Temperature $\tau > 0$:

$$p_\tau(x_t = v) = \frac{\exp(z_{t,v}/\tau)}{\sum_{v'} \exp(z_{t,v'}/\tau)}$$

$\tau \to 0$: greedy (argmax), $\tau \to \infty$: uniform.

Top-k: keep only the k highest-probability tokens.

Nucleus (top-p): keep smallest set $\mathcal{V}_p$ s.t. $\sum_{v \in \mathcal{V}_p} p(v \mid x_{<t}) \geq p$.

These strategies can be combined: e.g. top-p + temperature.

Pre-Training Objective

Causal Language Modeling (next-token prediction):

$$\mathcal{L}(\theta) = -\sum_{t=1}^{n} \log p_\theta(x_t \mid x_{<t})$$

where:

$$p_\theta(x_t \mid x_{<t}) = \text{softmax}\big(\boldsymbol{z}_t = \boldsymbol{h}_t^{(L)} W_E^\top\big)_{x_t}$$

$\boldsymbol{h}_t^{(L)}$: hidden state at position t, last transformer block L
$W_E \in \mathbb{R}^{|V| \times d}$: embedding matrix

Here $\boldsymbol{h}^{(L)}$ corresponds to $\boldsymbol{h}$ of the previous slides – we previously did not explicitly index layers for conciseness

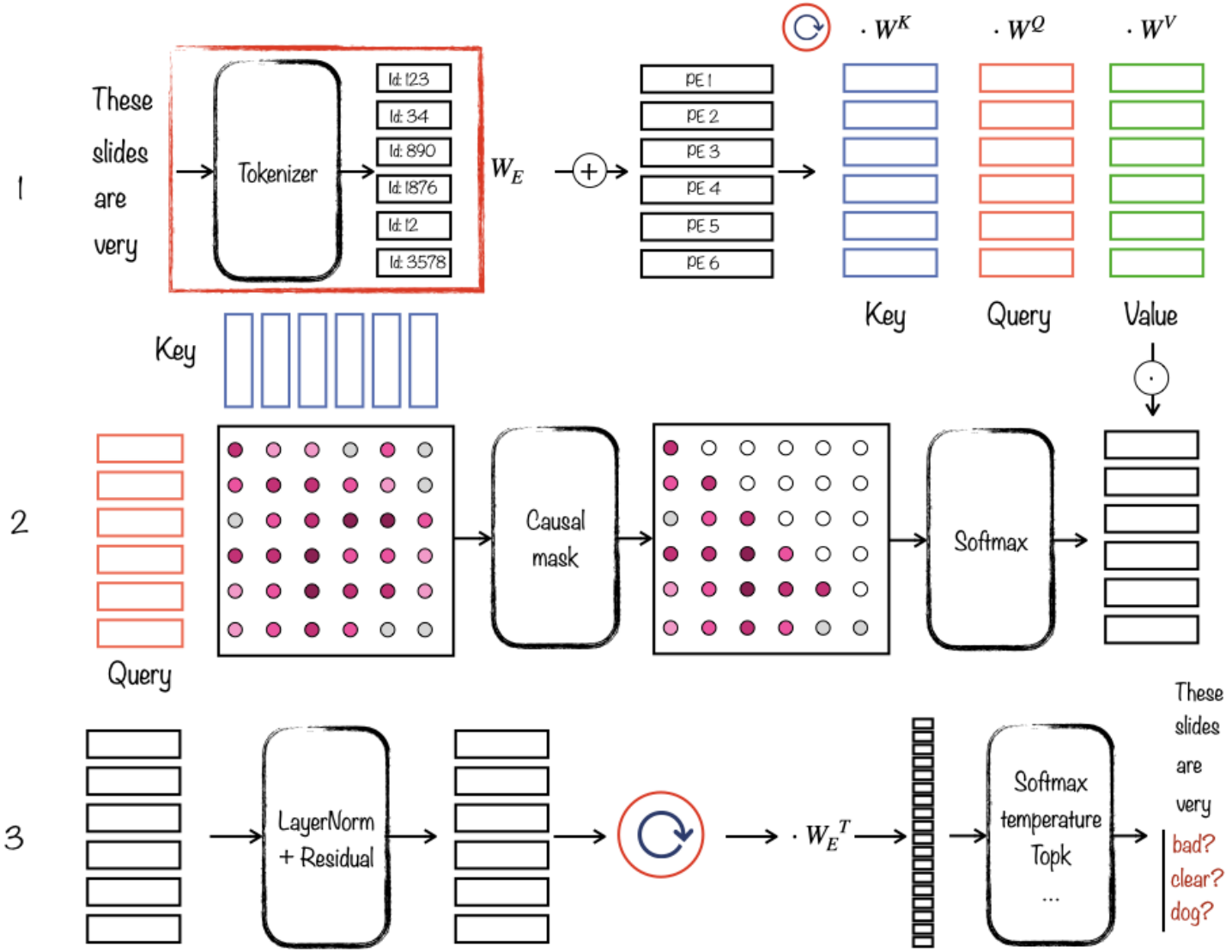
Wrap-up

Interactive Transformer Explainer

poloclub.github.io/transformer-explainer

Modern LLMs

Modern Tokenization



Modern Tokenization

Byte-level BPE (Radford et al., 2019 — GPT-2):

→ Operate on raw **bytes** (256 base symbols) instead of characters

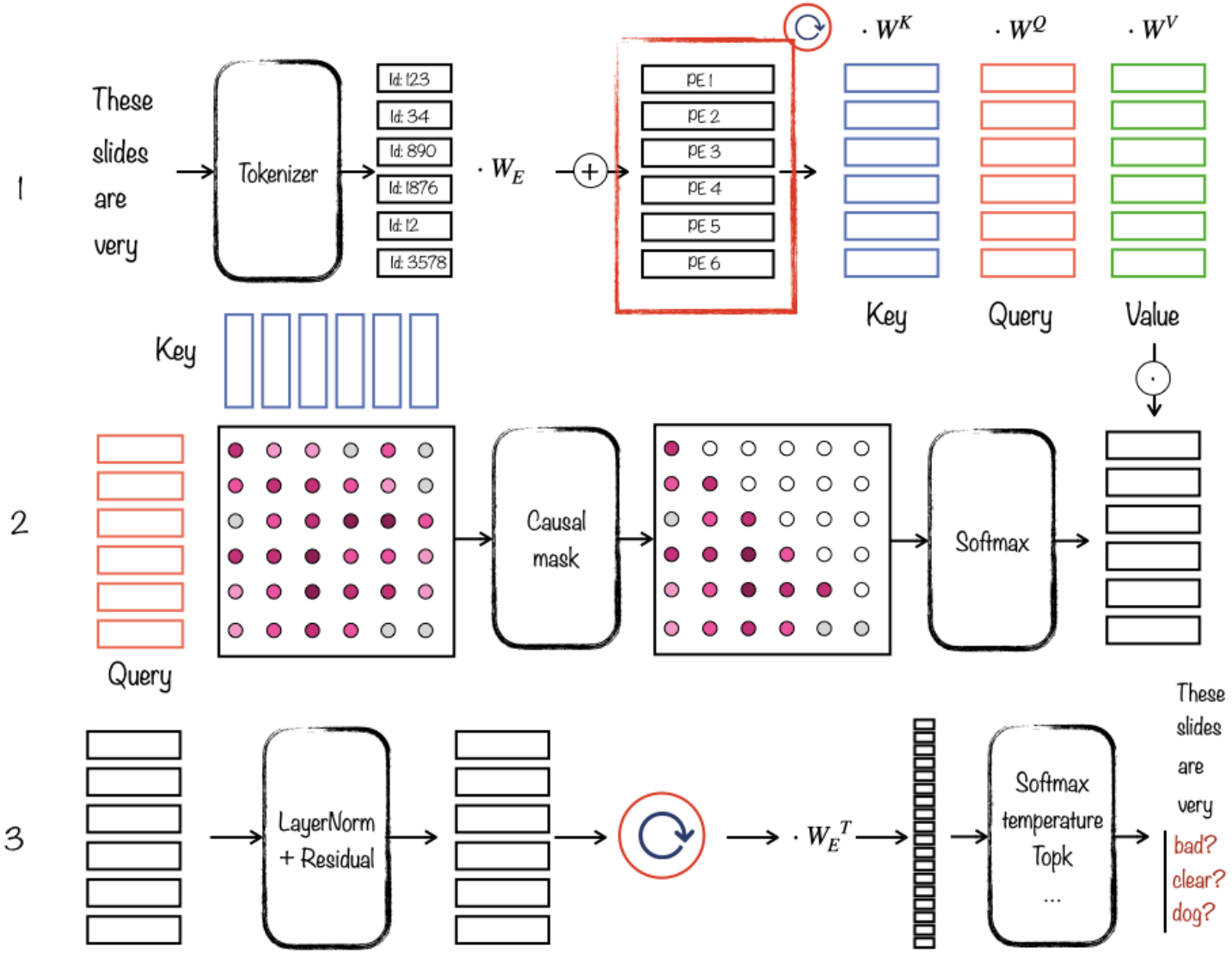
→ Can encode *any* text — no unknown tokens

→ Used by GPT-3/4/5, LLaMA, Claude

Model	**Tokenizer**	$\lvert V \rvert$
GPT-2 / GPT-3	Byte-level BPE	50K
GPT-4	tiktoken (Byte-level BPE)	100K
GPT-5	tiktoken (`o200k_base`)	200K
LLaMA 3	tiktoken (Byte-level BPE)	128K

tiktoken is OpenAI's tokenizer library (GPT-5 uses `o200k_base`).

Rotary Position Embeddings (RoPE)



Rotary Position Embeddings (RoPE)

Su et al. (2021): encode position via rotation in 2D subspaces.

For position m, apply rotation R_m to $\boldsymbol{q}$ and $\boldsymbol{k}$:

$$R_m = \text{diag}\big(R_{\theta_1,m},\ \ldots,\ R_{\theta_{d/2},m}\big)$$

where each block is a 2D rotation:

$$R_{\theta_i,m} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix}, \quad \theta_i = 10000^{-2i/d}$$

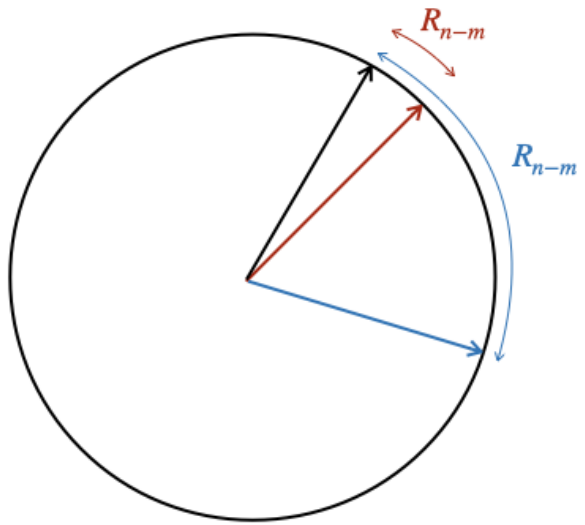
Key property:

$$(R_m \boldsymbol{q})^\top (R_n \boldsymbol{k}) = \boldsymbol{q}^\top R_{n-m}\, \boldsymbol{k}$$

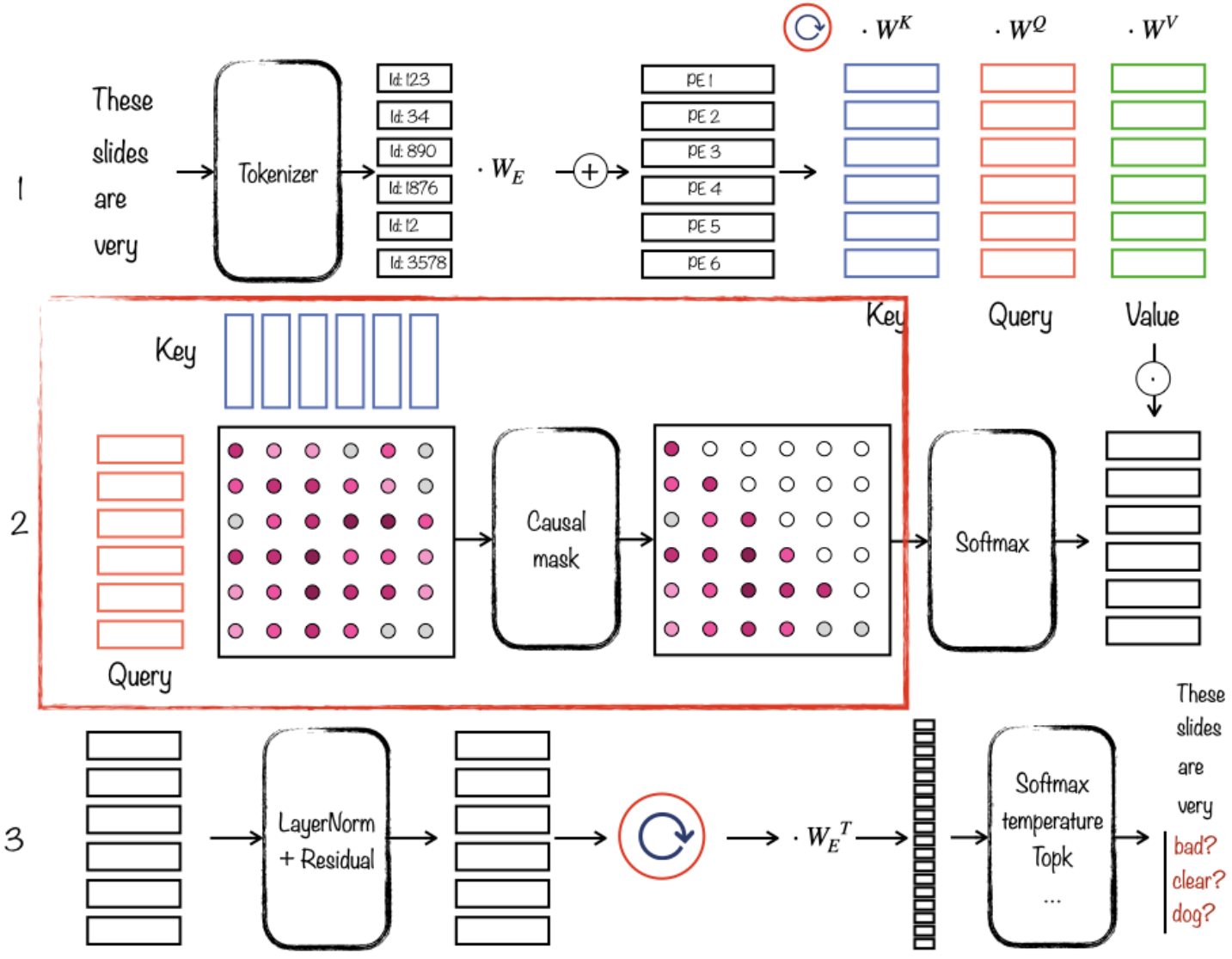
Rotary Position Embeddings (RoPE)

$$\mathbf{q}^T R_{n-m} \mathbf{k}$$

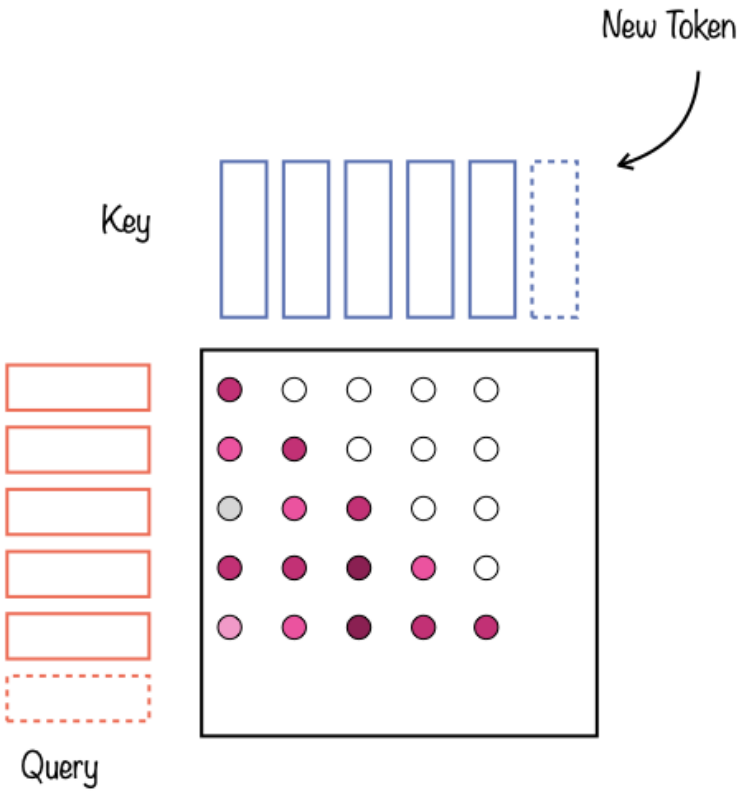
$n \approx m$ $\quad$ $n \ll m$



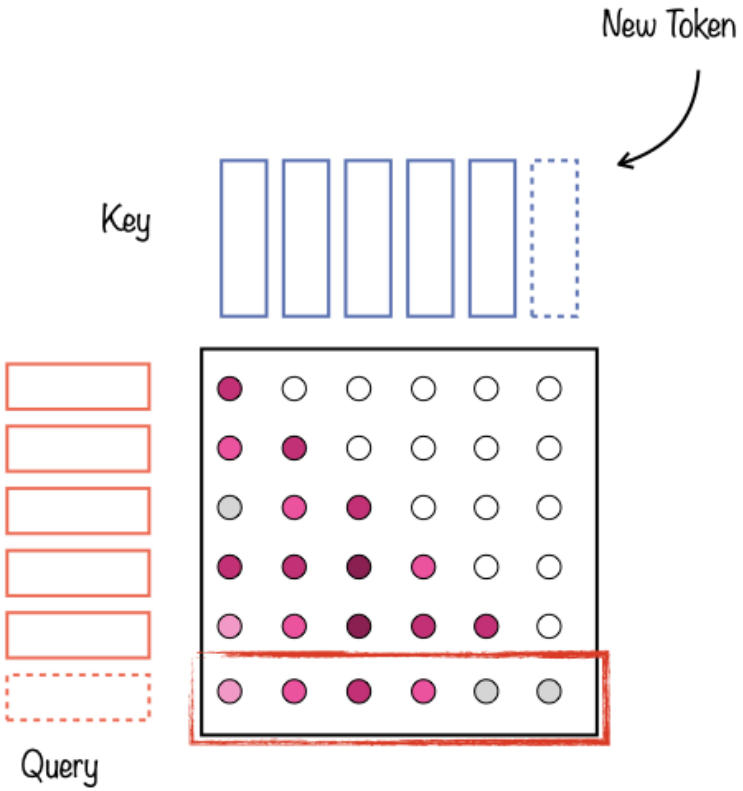
KV-Cache


1
These
slides
are
very
Tokenizer
Id: 123
Id: 34
Id: 890
Id: 1876
Id: 12
Id: 3578
$\cdot W_E$
PE 1
PE 2
PE 3
PE 4
PE 5
PE 6
$\cdot W^K$
$\cdot W^Q$
$\cdot W^V$
Key
Query
Value
2
Key
Query
Causal
mask
Softmax
3
LayerNorm
+ Residual
$\cdot W_E{}^T$
Softmax
temperature
Topk
...
These
slides
are
very
bad?
clear?
dog?

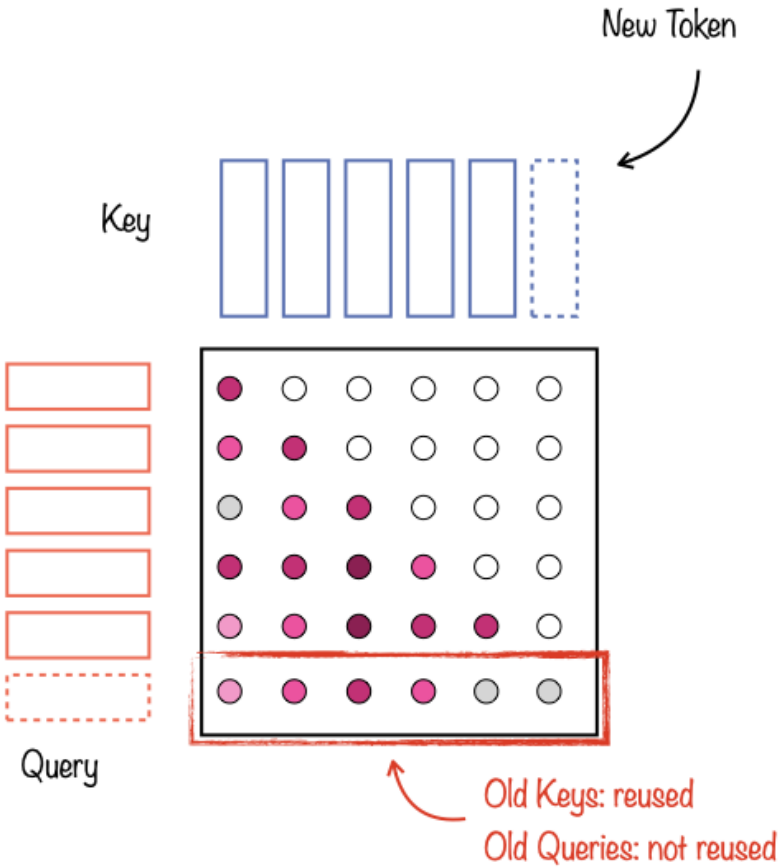
KV-Cache



KV-Cache



KV-Cache



KV-Cache

During autoregressive generation, tokens are produced one at a time.

Naive: recompute K, V for all previous tokens at each step $\rightarrow O(n^2)$.

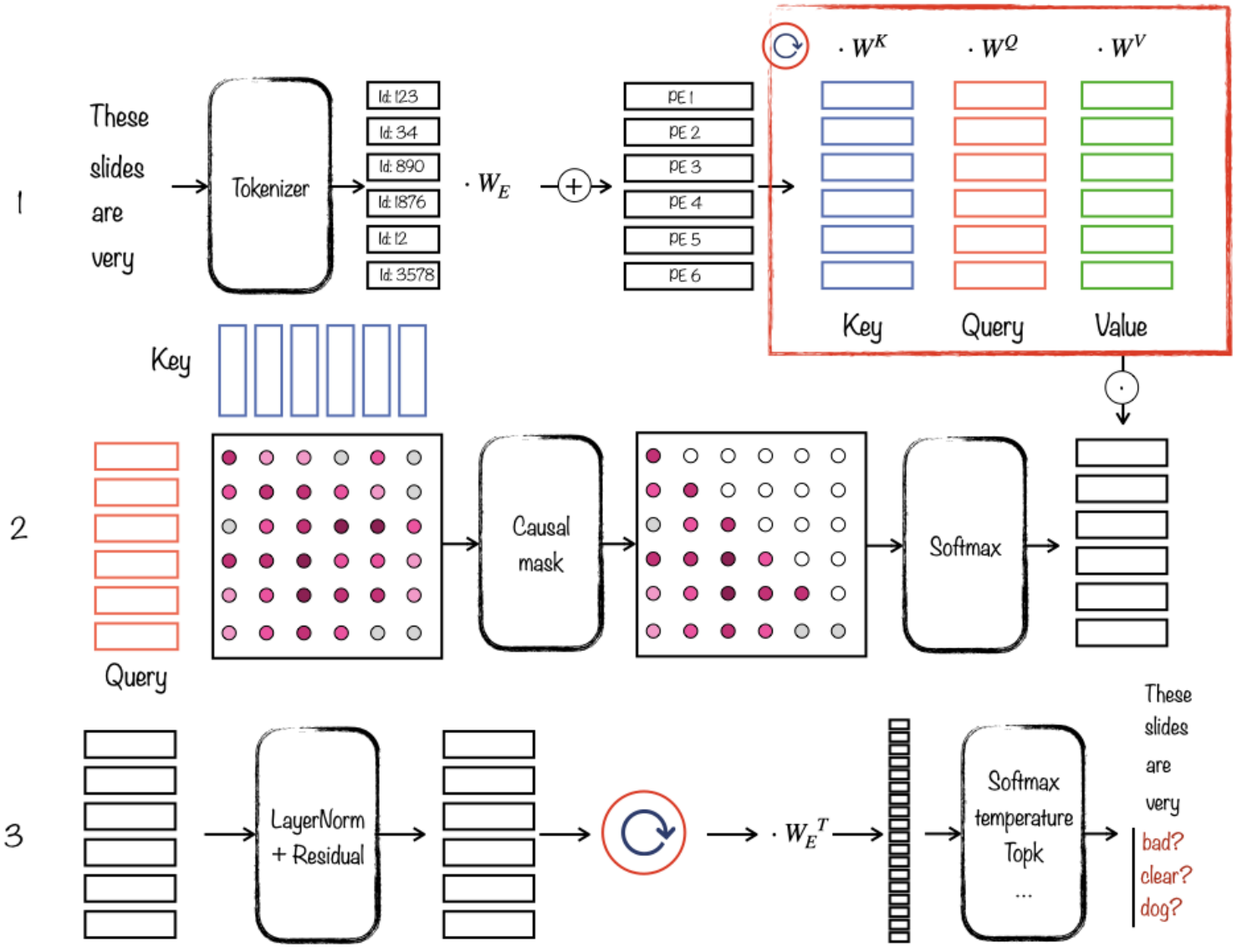
KV-Cache: store and append K_t, V_t from previous steps:

$$K_{\leq t} = \text{Concat}(K_{\leq t-1},\ \boldsymbol{x}_t W^K), \qquad \boldsymbol{q}_t = \boldsymbol{x}_t W^Q$$
$$V_{\leq t} = \text{Concat}(V_{\leq t-1},\ \boldsymbol{x}_t W^V)$$

$$\text{output}_t = \text{Attention}(\boldsymbol{q}_t,\ K_{\leq t},\ V_{\leq t})$$

Memory: $O(n \cdot d \cdot L)$ per sequence. GQA reduces this by sharing K/V heads.

Grouped Query Attention (GQA)



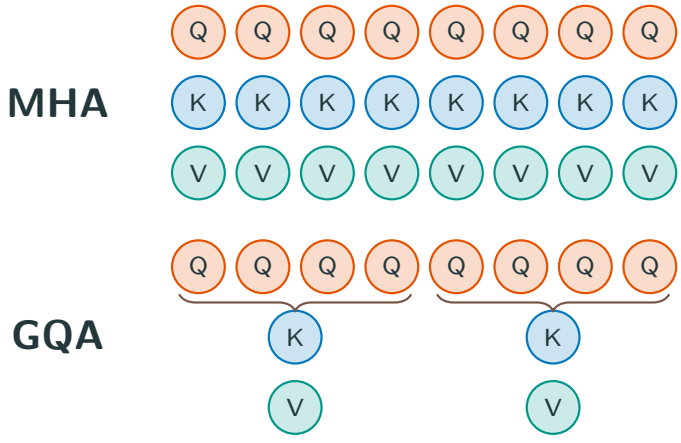
Grouped Query Attention (GQA)

Ainslie et al. (2023): share K/V heads across groups of Q heads.

→ During decoding, **KV-cache bandwidth/memory** is often the bottleneck

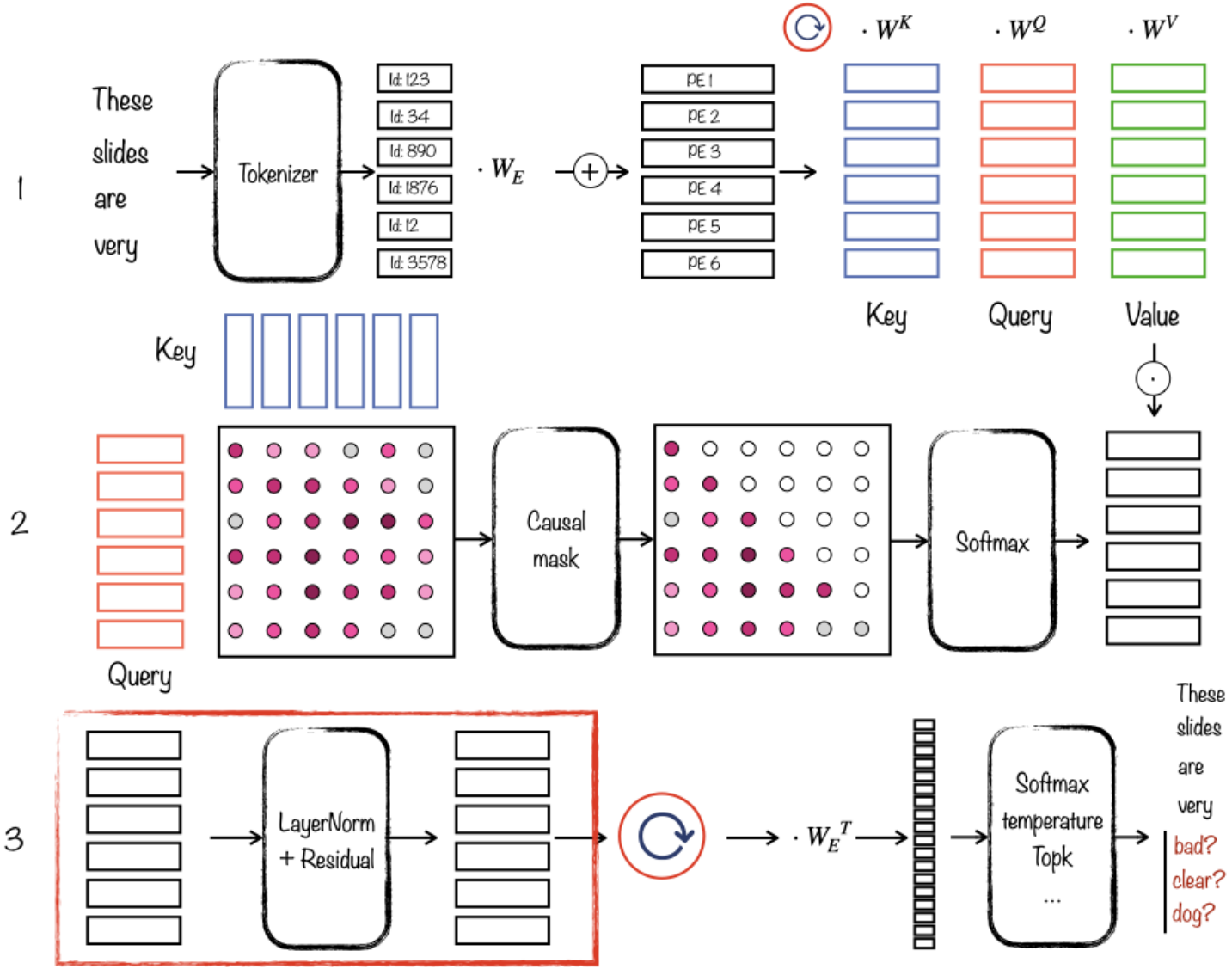
→ In MHA, each query head has its own K/V head ⇒ large cache

→ GQA keeps many Q heads but shares fewer K/V heads ⇒ faster inference



If h_q query heads use only h_{kv} K/V heads, KV-cache size drops by about h_q/h_{kv} (here: $8/2 = 4\times$ smaller than MHA).

LayerNorm → RMSNorm



LayerNorm $\rightarrow$ RMSNorm

Root Mean Square Layer Normalization (Zhang and Sennrich, 2019):

$$\text{RMSNorm}(\boldsymbol{h}) = \frac{\boldsymbol{h}}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot \boldsymbol{\gamma}$$

vs. LayerNorm:

No mean subtraction, no bias $\boldsymbol{\beta}$

Cheaper to compute, similar performance (huge advantage when scaling to trillions of parameters...)

Modern LLMs use **pre-norm** (normalize *before* sublayer):

$$\boldsymbol{h} = \boldsymbol{h} + \text{Layer}\big(\text{RMSNorm}(\boldsymbol{h})\big)$$

SwiGLU Activation

Shazeer (2020): use Swish in the gate to get a smoother, learnable filter. **Swish** (gate path):

$$\text{Swish}(x) = x \cdot \sigma(\beta x), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

SwiGLU (used instead of standard MLP):

$$\boxed{\text{SwiGLU}(\boldsymbol{h}) = \left(\text{Swish}(\boldsymbol{h}W_1) \odot \boldsymbol{h}W_3\right) W_2}$$

Interpretation:

→ $\boldsymbol{h}W_3$: candidate features

→ $\text{Swish}(\boldsymbol{h}W_1)$: soft gate (how much to keep per feature)

→ $\odot$: feature-wise modulation

In practice, SwiGLU often improves perplexity/quality at similar compute.

SwiGLU Activation

Shazeer (2020): use Swish in the gate to get a smoother, learnable filter.

Swish (gate path):

$$\text{Swish}(x) = x \cdot \sigma(\beta x), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

SwiGLU (used instead of standard MLP):

$$\boxed{\text{SwiGLU}(\boldsymbol{h}) = \left(\text{Swish}(\boldsymbol{h}W_1) \odot \boldsymbol{h}W_3\right) W_2}$$

Interpretation:

→ $\boldsymbol{h}W_3$: candidate features

→ $\text{Swish}(\boldsymbol{h}W_1)$: soft gate (how much to keep per feature)

Summary

Key Takeaways

Motivation

- Any text is its own label — no annotation needed
- Loss $\propto N^{-\alpha}$: scale params & tokens together
- Billions of web pages become free training data

Decoder Transformer

- Causal mask: each token sees only past tokens
- $\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$
- Multi-head: diverse patterns, same compute
- Temperature / top-p tune generation diversity

Modern LLMs

- RoPE: relative position via rotation
- RMSNorm + SwiGLU: efficiency & gated MLP
- KV-Cache: store K/V for $O(n)$ decoding
- GQA: shared K/V $\Rightarrow$ smaller KV-cache

One same architecture trained on raw text at scale for GPT, LLaMA, and Claude.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *CoRR*, abs/2305.13245, 2023. doi: 10.48550/ARXIV.2305.13245. URL https://doi.org/10.48550/arXiv.2305.13245.

Lei Jimmy Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *CoRR*, abs/1607.06450, 2016. URL http://arxiv.org/abs/1607.06450.

References ii

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*, 2015. URL https://arxiv.org/abs/1409.0473.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *CoRR*, abs/2203.15556, 2022. doi: 10.48550/ARXIV.2203.15556. URL https://doi.org/10.48550/arXiv.2203.15556.

References iii

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *CoRR*, abs/2001.08361, 2020. URL https://arxiv.org/abs/2001.08361.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1007. URL https://aclanthology.org/P18-1007.

References iv

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018. URL https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

References v

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162. URL https://aclanthology.org/P16-1162.

Noam Shazeer. GLU variants improve transformer. *CoRR*, abs/2002.05202, 2020. URL https://arxiv.org/abs/2002.05202.

Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *CoRR*, abs/2104.09864, 2021. URL https://arxiv.org/abs/2104.09864.

References vi

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971, 2023. doi: 10.48550/ARXIV.2302.13971. URL https://doi.org/10.48550/arXiv.2302.13971.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL http://arxiv.org/abs/1706.03762.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. *CoRR*, abs/1910.07467, 2019. URL http://arxiv.org/abs/1910.07467.